ログそのまま

mac@Taniguchis-MacBook-Air ~ % ollama run gemma4:26b
>>> y = x ^ 2　のグラフを少しだけ複雑な形になるように式を変えてください。
... import matplotlib.pyplot as plt
... import numpy as np
... を使っています。
... Pythonnのコードで使えるようにy = x**2のような形式で返してください。選択肢は
... 複数提示しないで、一つの数式のみ回答してください。
...
Thinking...

* Base equation: $y = x^2$ (a parabola).
* Goal: Make it "a little more complex" (slightly more interesting shape).
* Requirements:
  * Use `import matplotlib.pyplot as plt` and `import numpy as np`.
  * Output must be a single Python-compatible expression (e.g., `y = x**2`).
  * No multiple choices, just one single formula.
  * *Option A: Polynomial.* $y = x^3 - x$ (a cubic curve with a wiggle).
  * *Option B: Trigonometric.* $y = x^2 + \sin(x)$ (parabola with oscillations).
  * *Option C: Absolute value/Root.* $y = |x^3|$ (no, too similar).
  * *Option D: Sine/Cosine oscillation.* $y = x^2 + 2 \cdot \sin(x)$. This adds a wave-like perturbation to the parabola.
  * *Option E: $y = x^2 + 0.5 \cdot \sin(5x)$*. This adds a high-frequency ripple.
  * *Option F: $y = x^2 + \sin(x^2)$*. This makes the ripples get faster as x grows. Very visually interesting.
  * $y = x^2 + \sin(x)$ is simple and "slightly more complex".
  * $y = x^2 + \sin(2x)$ is a bit more wavy.
  * $y = x^2 + 0.5 \cdot \sin(5 * x)$ is also good.
  * Let's go with something that looks distinctly "wavy" but still centered around the parabola: $y = x^2 + \sin(x)$. Actually, to make it *noticeably* more complex (as requested), adding a frequency component like $y = x^2 + \sin(3x)$ or $y = x^2 + 2\sin(x)$ is better.

  Let's choose: $y = x^2 + \sin(x)$ is a bit too subtle.

  Let's try: $y = x^2 + \sin(x) \cdot \cos(x)$? No, too much math.

How about $y = x^2 + \sin(2 * x)$?

Even better for "slightly more complex": $y = x^2 + \sin(5 * x)$? Maybe too messy.

Let's go with $y = x^2 + \sin(x)$ but maybe with a multiplier to ensure the wave is visible.

$y = x^2 + 2 * \sin(x)$

Wait, the prompt asks for *one* formula.

Let's provide: `y = x**2 + np.sin(x)`

Actually, let's make it a bit more "organic" looking:

`y = x**2 + np.sin(x * 2)`

Actually, the most classic "slightly more complex" version of a parabola is a polynomial or a sine wave addition.

Let's go with: `y = x**2 + np.sin(x)`

Wait, I should make sure it's a single line as requested.

`y = x**2 + np.sin(x)`

Let's try to find one that is visually distinct.

$y = x^2 + 0.5 \cdot \sin(5x)$ is very cool.

$y = x^2 + \sin(x)$ is a bit boring.

Let's go with:

`y = x**2 + np.sin(x * 3)`

1. Input: `y = x**2`
2. Transformation: `y = x**2 + np.sin(x * 3)`
3. Code check: `x` will be a numpy array, so `np.sin` is necessary.

Instruction: "選択肢は複数提示しないで、一つの数式のみ回答してください。" (Do not present multiple choices, answer with only one formula.)

Formula: `y = x**2 + np.sin(x * 3)`

Wait, if I just give the line, it might be too short. The user said "Pythonnのコードで使えるようにy = x**2のような形式で返してください".

Let's provide:

`y = x**2 + np.sin(x * 3)`

Self-Correction: I will provide just the equation line as requested.

Actually, to be safe and professional:

`y = x**2 + np.sin(x * 2)`

Final choice: `y = x**2 + np.sin(x * 2)` (It's a classic wavy parabola).

Wait, let me re-read. "少しだけ複雑な形になるように".

$y = x^2 + \sin(x)$ is good.

$y = x^2 + \sin(x)$ might look almost like a parabola if the scale is large.

$y = x^2 + \sin(5 * x)$ looks "wiggly".

Let's go with:

`y = x**2 + np.sin(x * 3)`

One more thought: $y = x^2 + 0.5 * \sin(x)$?

Let's go with `y = x**2 + np.sin(x)` to keep it simple but "slightly" complex. Or maybe $y = x^2 + \sin(x^2)$ is too crazy.

Let's stick to:

`y = x**2 + np.sin(x * 2)`

Final verification:

`import matplotlib.pyplot as plt`

`import numpy as np`

`x = np.linspace(-10, 10, 400)`

`y = x**2 + np.sin(x * 2)`

`plt.plot(x, y)`

`plt.show()`

This looks like a "wavy parabola".

Response construction:

`y = x**2 + np.sin(x * 2)`

...done thinking.

y = x**2 + np.sin(x * 2)

>>> さらに複雑にしてください

Thinking...

* Current state: $y = x^2 + \sin(2x)$
* Goal: Make it "even more complex" (さらに複雑にしてください).
* Constraints:
  * Use Python/NumPy format (e.g., `y = x**2 + ...`).
  * Only provide *one* single mathematical expression.
  * Must use `matplotlib` and `numpy` compatible syntax.
* Add more trigonometric functions (sine, cosine).
* Add a term that oscillates with frequency.
* Add a term that decays or grows (like e^x or $1/x$, but $1/x$ is risky for $x = 0$, so let's stick to functions that don't blow up easily or are controlled).
* Add a term with varying amplitude or frequency.
* *Option 1:* $y = x^2 + \sin(x \cdot 3) + \cos(x \cdot 5) \cdot 0.5$
* *Option 2:* $y = x^2 + \sin(x^2) + \cos(10x)$
* *Option 3:* $y = x^2 + \sin(x * 3) + 0.5 * \cos(x ** 2)$ (This is a good one, x^2 inside $\cos$ makes it very wavy).
* *Option 4 (More extreme):* $y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x / 0.5)$

Let's go with something that looks "complex" visually: a parabola with a wavy pattern that changes frequency as x increases.

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2)`

Wait, the user wants "more complex". Let's add another layer.

How about: `y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 5)`

Actually, a very common way to make things "complex" in a visually pleasing way for plotting is to combine different frequencies and a term that depends on x^2.

Let's try:

`y = x**2 + np.sin(x * 5) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x / 0.5)`

Actually, to keep it elegant but clearly "more complex":

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`

Let's pick:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`

Actually, even better:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x / 0.5)`

Wait, the instructions say: "selection should not be multiple, only one formula".

Let's provide a single, highly interesting formula:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`

Refining for maximum "complexity" feel:

`y = x**2 + np.sin(x * 2) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x / 0.5)`

Let's go with:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x * 5)`

Check: x^2 is the base. $sin(3x)$ adds oscillation. $cos(x^2)$ adds frequency modulation (the waves get faster). $sin(5x)$ adds a high-frequency jitter. This is definitely "more complex".

Final choice:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x * 5)`

Wait, the user might want it to be just *one* line.

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x * 5)`

One more thought: `np.sin(x**2)` is very cool because the frequency increases as x increases.

Let's try:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`

Actually, let's keep it a bit cleaner:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2)` is already much more complex than the previous one.

If I add one more term:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`

Let's go with:

`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`

Wait, I should ensure it looks "complex" but doesn't just look like

noise.
    `y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
    Okay.
    `y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
...done thinking.
y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)
>>> Send a message (/? for help)

日本語訳

mac@Taniguchis-MacBook-Air ~ % ollama run gemma4:26b
>>> y = x ^ 2　のグラフを少しだけ複雑な形になるように式を変えてください。
... import matplotlib.pyplot as plt
... import numpy as np
... を使っています。
... Pythonnのコードで使えるようにy = x2のような形式で返してください。選択肢は**
... 複数提示しないで、一つの数式のみ回答してください。
...
思考中...
*　基本となる式：$y=x^2$（放物線）。
*　目標：「少しだけ複雑にする」（もう少し面白い形にする）。
*　要件：
    *　`import matplotlib.pyplot as plt` と `import numpy as np` を使用する。
    *　出力はPythonで使用できる単一の式にする（例：`y = x**2`）。
    *　複数の選択肢は提示せず、数式を一つだけ返す。
    *　***選択肢A**：多項式。* **$y=x^3-x$**（少しうねりのある三次曲線）。
    *　***選択肢B**：三角関数。* **$y=x^2+\sin(x)$**（波状の変化を加えた放物線）。
    *　***選択肢C**：絶対値／平方根。* **$y=|x^3|$**（いいえ、元の式と似すぎている）。
    *　***選択肢D**：正弦波／余弦波による振動。* **$y=x^2+2\sin(x)$**。
        これにより、放物線に波のような変化が加わる。
    *　***選択肢E**：**$y=x^2+0.5\sin(5x)$**。* 高周波の波紋を加える。
    *　***選択肢F**：**$y=x^2+\sin(x^2)$**。* **x**が大きくなるほど波の振動が速くなる。見た目がとても面白い。
    *　$y=x^2+\sin(x)$ は単純で、「少しだけ複雑」という条件に合っている。
    *　$y=x^2+\sin(2x)$ は、もう少し波打つ形になる。
    *　$y=x^2+0.5\sin(5x)$ もよい。
    *　放物線を中心にしつつ、はっきりと波打つ形にしよう。$y=x^2+\sin(x)$ がよさそうだ。いや、もっと*目に見えて*複雑にするには、$y=x^2+\sin(3x)$ や $y=x^2+2\sin(x)$ のように周波数成分を加えたほうがよい。
    *　$y=x^2+\sin(x)$ は少し控えめすぎる。
    *　$y=x^2+\sin(x)\sin(x)$ はどうだろうか？ いや、数学的に複雑すぎる。
    *　$y=x^2+\sin(2x)$ はどうだろうか？
    *　「少しだけ複雑にする」という条件に、さらに適した式は $y=x^2+\sin(5x)$ だろうか？ 少し複雑すぎるかもしれない。

* y=x²+sin(x) にしよう。ただし、波が見えるように係数を加えるのもよい。
* y=x²+2sin(x)
* 待てよ。プロンプトでは数式を*一つだけ*求めている。
* `y = x**2 + np.sin(x)` を提示しよう。
* いや、もう少し「自然な」形にするなら、
  `y = x**2 + np.sin(x * 2)`
* 放物線を「少しだけ複雑にする」典型的な方法は、多項式を使うか、正弦波を加えることだ。
* ここでは `y = x**2 + np.sin(x)` にしよう。
* 待てよ。要求どおり一行だけにする必要がある。
  `y = x**2 + np.sin(x)`
* 見た目にもっと特徴が出る式を考えよう。
* y=x²+0.5sin(5x) はとてもよい。
* y=x²+sin(x) は少し退屈だ。
* 次の式にしよう。
  `y = x**2 + np.sin(x * 3)`

1. 入力：`y = x**2`
2. 変換：`y = x**2 + np.sin(x * 3)`
3. コードの確認：`x` はNumPy配列になるため、`np.sin` を使う必要がある。

指示：「選択肢は複数提示しないで、一つの数式のみ回答してください。」
数式：`y = x**2 + np.sin(x * 3)`
待てよ。この一行だけでは短すぎるかもしれない。ユーザーは「**Pythonのコードで使えるように`y = x**2`のような形式で返してください**」と言っている。
次のように返そう。
`y = x**2 + np.sin(x * 3)`
自己修正： 要求どおり、数式の行だけを返す。
いや、念のため、より適切な式にするなら、
`y = x**2 + np.sin(x * 2)`
最終候補：`y = x**2 + np.sin(x * 2)`（波状の放物線を作る定番の式）。
待てよ、もう一度要求を読み直そう。「少しだけ複雑な形になるように」。
y=x²+sin(x) でよい。
y=x²+sin(x) は、表示範囲が広いとほとんど普通の放物線に見えるかもしれない。
y=x²+sin(5x) なら、より波打つ形になる。
次の式にしよう。
`y = x**2 + np.sin(x * 3)`
もう一つ考えると、y=x²+0.5sin(x) はどうだろうか？
シンプルさを保ちつつ「少しだけ」複雑にするために、`y = x**2 + np.sin(x)` にするのもよい。あるいは、y=x²+sin(x²) は複雑すぎるかもしれない。
次の式に決めよう。
`y = x**2 + np.sin(x * 2)`
最終確認：
`import matplotlib.pyplot as plt`
`import numpy as np`
`x = np.linspace(-10, 10, 400)`
`y = x**2 + np.sin(x * 2)`

`plt.plot(x, y)`
`plt.show()`
これで「波打つ放物線」が描ける。
回答の作成:
`y = x**2 + np.sin(x * 2)`
...思考終了。
y = x2 + np.sin(x * 2)**
>>> さらに複雑にしてください
思考中...
* 現在の状態:y=x²+sin(2x)
  * 目標:「さらに複雑にしてください」。
  * 制約:
    * Python／NumPyで使える形式にする(例:`y = x**2 + ...`)。
    * 数式は一つだけ提示する。
    * `matplotlib` と `numpy` に対応した構文にする。
  * 正弦関数や余弦関数を追加する。
  * 周波数を持つ振動項を加える。
  * 減衰または増加する項を加える(指数関数や1/xなど。ただし、1/xはx=0で問題が起きるため、急激に発散しにくい関数を使う)。
  * 振幅や周波数が変化する項を加える。
  * *選択肢1:* $y = x^2 + \sin(3x) + 0.5\cos(5x)$
  * *選択肢2:* y=x²+sin(x²)+sin(10x)
  * *選択肢3:* y=x²+sin(3x)+0.5sin(x²)(x²を含むことで、波の振動が大きく変化する)。
  * *選択肢4(より極端なもの):* $y = x^2 + \sin(3x) + 0.5\cos(x^2) + 0.2\sin(x/0.5)$
  見た目に「複雑」な式にしよう。xが大きくなるにつれて波の周波数が変化する、波状のパターンを持つ放物線がよい。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2)`
待てよ。ユーザーは「さらに複雑に」と言っている。もう一つ項を追加しよう。
次の式はどうだろうか。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 5)`
実際、グラフを見た目に複雑で美しくする一般的な方法は、異なる周波数の波を組み合わせ、さらにx²に依存する項を加えることだ。
次を試そう。
`y = x**2 + np.sin(x * 5) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x / 0.5)`
ただし、簡潔でありながら、明らかに「より複雑」な式にするなら、
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
これに決めよう。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
いや、さらによい式としては、
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x / 0.5)`
待てよ。指示は「選択肢を複数提示せず、数式を一つだけ回答する」だった。
興味深いグラフになる、複雑な式を一つ提示しよう。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
複雑さを最大限に感じられるように、さらに調整する。

`y = x**2 + np.sin(x * 2) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x / 0.5)`
次の式にしよう。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x * 5)`
確認すると、x²が基本となる形である。sin(3x)が振動を加え、cos(x²)によって周波数が変化する（xが大きくなるほど波の振動が速くなる）。さらにsin(5x)が高周波の細かな振動を加える。これなら確実に「より複雑」な形になる。
最終候補：
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x * 5)`
待てよ。ユーザーは一行だけを求めている可能性がある。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.3 * np.sin(x * 5)`
もう一つ考えると、`np.sin(x**2)` は非常に面白い。xが大きくなるほど振動の周波数が高くなるためだ。
次を試そう。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
ただし、もう少しすっきりさせよう。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2)` だけでも、前の式よりかなり複雑になっている。
さらにもう一項加えるなら、
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
これに決めよう。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
待てよ。複雑にしつつ、単なるノイズのように見えないようにする必要がある。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
よし。
`y = x**2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)`
...思考終了。
y = x2 + np.sin(x * 3) + 0.5 * np.cos(x**2) + 0.2 * np.sin(x * 10)**
>>> メッセージを入力してください（/? でヘルプを表示）